

Software Testing 1st Unit Notes

1. Overview of Software Evolution

Software evolution refers to the process of developing, maintaining, and updating software over time to meet changing user requirements and environments.

Key Points:

- Software is not static; it continuously evolves due to:
 - Changing user needs
 - Technological advancements
 - Bug fixes and performance improvements
- Evolution starts after deployment and continues throughout the lifecycle.

Lehman's Laws of Software Evolution:

1. **Continuing Change** – Software must evolve or becomes useless.
2. **Increasing Complexity** – Complexity increases unless actively reduced.
3. **Self-Regulation** – Evolution process is self-regulated.
4. **Conservation of Organizational Stability** – Development rate remains constant.
5. **Conservation of Familiarity** – Changes are gradual.
6. **Continuing Growth** – Functionality must increase.
7. **Declining Quality** – Quality decreases without maintenance.
8. **Feedback System** – Evolution depends on feedback.

Types of Software Maintenance:

- **Corrective** – Fix errors
- **Adaptive** – Adjust to environment changes
- **Perfective** – Improve performance
- **Preventive** – Avoid future issues

Example:

A mobile app updating its UI and fixing bugs regularly.

2. SDLC (Software Development Life Cycle)

SDLC is a structured process used to develop software systematically.

Phases:

1. **Requirement Analysis**
 - Collect user requirements
 - Create SRS document
2. **System Design**
 - High-Level Design (HLD)
 - Low-Level Design (LLD)
3. **Implementation (Coding)**
 - Convert design into code
4. **Testing**
 - Identify and fix defects
5. **Deployment**
 - Release software to users
6. **Maintenance**
 - Update and improve software

Requirement Analysis

What is Requirement Analysis?

Requirement Analysis is the first and most critical phase of SDLC, where we identify, gather, analyze, and document user needs.

- What does the user want?
- What problem should the system solve?

How Requirements are Collected?

1. Requirement Gathering Techniques:

- **Interviews** → Direct interaction with stakeholders
- **Questionnaires** → For large groups
- **Observation** → Study current system
- **Workshops** → Group discussions
- **Prototyping** → Show early model to users

Types of Requirements:

1. **Functional Requirements**
 - What system should do
 - Example: “User can login”
2. **Non-Functional Requirements**
 - Performance, security, usability
 - Example: “System should respond within 2 seconds”

2. System Design

What is System Design?

System Design converts requirements into system architecture.

- How will the system be built?

Types of Design:

High-Level Design (HLD)

What is HLD?

Describes overall system architecture.

Includes:

- System modules
- Data flow
- Technology stack
- Database design (high-level)

Example:

E-commerce system modules:

- User Module
- Payment Module
- Order Module

Output of HLD:

- Architecture diagrams
- Module structure

Low-Level Design (LLD)

What is LLD?

Detailed design of each module.

Includes:

- Algorithms
- Class diagrams

- Database tables
- Function definitions

Difference Between HLD and LLD:

HLD	LLD
High-level overview	Detailed design
Architecture	Logic
System-level	Module-level

Importance of Design:

- Reduces coding complexity
- Improves maintainability
- Helps in early error detection

3. Implementation (Coding)

What is Implementation?

The phase where design is converted into actual code.

Activities:

- Writing code using programming languages
- Following coding standards
- Code documentation

How Coding is Done?

- Divide work into modules
- Assign to developers
- Use version control (Git)

Best Practices:

- Clean code
- Proper naming conventions
- Reusability
- Commenting

Output:
Source Code

Example:

Design → Login module
Coding → Write Java/Python code for login functionality

4. Testing

What is Testing?

Testing is the process of identifying defects and ensuring software works correctly.

Objectives:

- Detect errors
- Ensure quality
- Validate requirements

Types of Testing:

1. Unit Testing

- Test individual components

2. Integration Testing

- Test combined modules

3. System Testing

- Test complete system

4. Acceptance Testing

- Done by users

Testing Process:

1. Test Planning
2. Test Case Design
3. Execution
4. Defect Reporting
5. Retesting

Output:

Bug reports + Tested software

Importance:

- Improves reliability
- Reduces failures in production

5. Deployment

What is Deployment?

Deployment is the process of

Activities:

- Installation
- Configuration
- Data migration
- User training

Types of Deployment:

1. **Direct Deployment**
 - Replace old system completely
2. **Parallel Deployment**
 - Old and new run together
3. **Phased Deployment**
 - Release in parts
4. **Pilot Deployment**
 - Release to limited users

Output:

Live system

Example:

Launching a mobile app on Play Store

6. Maintenance

What is Maintenance?

Maintenance is the process of modifying and updating software after deployment.

Types of Maintenance:

1. Corrective

- Fix bugs

2. Adaptive

- Adjust to environment changes

3. Perfective

- Improve performance

4. Preventive

- Avoid future issues

SDLC FLOW SUMMARY (for diagram in exam)

You can draw this:

```
Requirement Analysis
      ↓
System Design (HLD + LLD)
      ↓
Implementation (Coding)
      ↓
Testing
      ↓
Deployment
      ↓
Maintenance
```

SDLC Models:

- Waterfall Model
- Agile Model
- Spiral Model
- V-Model

Advantages:

- Improves quality
- Ensures systematic development
- Reduces risk

Example:

Banking software development follows SDLC phases strictly.

3. Testing Process

Software testing process ensures **quality and reliability**.

Steps:

1. **Test Planning**
 - Define scope, objectives
2. **Test Case Design**
 - Create test scenarios
3. **Test Environment Setup**
 - Prepare system for testing
4. **Test Execution**
 - Run test cases
5. **Defect Reporting**
 - Log bugs
6. **Test Closure**
 - Evaluate testing results

Types:

- Manual Testing
- Automated Testing

Importance:

- Ensures software correctness
- Improves reliability
- Reduces maintenance cost

Terminologies in Software Testing

1. Error

An Error is a human mistake made during software development.

- Occurs due to misunderstanding, wrong logic, or incorrect requirements.
- Happens at design or coding stage.

Example:

A programmer writes $a + b$ instead of $a - b$.

2. Fault (Bug/Defect)

A Fault is the incorrect code or logic in the program caused by an error.

- It is the manifestation of an error in code.
- Found during testing.

Example:

Wrong formula written in code for calculating interest.

3. Failure

A Failure occurs when the system does not perform as expected.

- It happens when a fault is executed.
- Visible to the user.

Example:

Bank app shows incorrect balance → system failure.

Relation:

Error → Fault → Failure

4. Verification

Verification answers: “Are we building the product, right?”

- Checks whether the software is developed according to specifications.
- Done without executing code (static process).

Techniques:

- Reviews
- Inspections
- Walkthroughs

Example:

Checking SRS document correctness.

5. Validation

Validation answers: “Are we building the right product?”

- Checks whether the software meets user requirements.
- Done by executing code (dynamic process).

Techniques:

- Testing
- User acceptance testing

Example:

Checking if login system works as user expects.

6. Difference Between Verification and Validation

Basis	Verification	Validation
Purpose	Check correctness of process	Check correctness of product
Question	Are we building it right?	Are we building the right product?
Type	Static	Dynamic
Execution	No code execution	Code execution required
Example	Review of design	Running test cases

7. Test Case

A Test Case is a set of conditions used to test a specific functionality.

It includes:

- Input data
- Execution steps
- Expected output
- Actual result

Example:

- Input: Username & Password
- Expected Output: Successful login

8. Test Suite

A Test Suite is a collection of multiple test cases.

- Used to test a complete module/system.
- Helps in organized testing.

Example:

All test cases related to login, signup, and password reset.

9. Test

A Test is the process of executing software to find defects.

- It checks whether the system behaves as expected.
- Can be manual or automated.

Example:

Running a login test to check valid/invalid credentials.

10. Test Oracle

A Test Oracle is a mechanism used to determine whether a test has passed or failed.

- It provides the expected result for comparison.

Types:

- Human oracle (manual checking)
- Automated oracle
- Document-based oracle (SRS)

Example:

Expected result: “Login successful”

If output matches → Test passed.

Impracticality of Testing All Data

Introduction

In software testing, it is impossible to test all possible input data for a program. This concept is known as the Impracticality of Testing All Data.

Even for a simple program, the number of possible inputs can be extremely large or even infinite.

Why Testing All Data is Impractical?

1. Huge Input Domain

- Programs can accept a very large range of inputs.
- Example:
If a system accepts a number from **1 to 1,00,000**, then there are 1,00,000 test cases.

Testing each value is time-consuming.

2. Infinite Input Possibilities

- Some inputs are continuous (like real numbers).

- Example:
Values between 0 and 1 can be infinite (0.1, 0.01, 0.001, ...)

Impossible to test all such values.

3. Combination Explosion

- When multiple inputs exist, combinations increase exponentially.

Example:

If a login system has:

- 100 usernames
- 100 passwords

Total combinations = $100 \times 100 = 10,000$ test cases

4. Time Constraint

- Software projects have deadlines.
- Testing every input would take too much time.

5. Cost Constraint

- More test cases → more resources → higher cost.
- Not feasible for organizations.

6. Resource Limitation

- Limited testers, tools, and infrastructure.
- Cannot handle exhaustive testing.

Example

Consider a simple function:

```
if (age >= 18 && age <= 60)
    eligible
```

- Possible values of age: 0 to 100 (or more)
- Testing all values is unnecessary and impractical.

Instead, we test:

- Boundary values (17, 18, 60, 61)
- Representative values

Verification

Verification is the process of checking whether the software is being built correctly according to specifications.

It answers:

“Are we building the product right?”

Key Characteristics:

- Static process (no execution required)
- Done during development phases
- Focus on documents, design, and code

Objectives:

- Ensure correctness of each phase output
- Detect errors early
- Improve software quality

◇ 2. Verification Methods

Main Methods:

1. Reviews

- Manual examination of documents/code
- Types:
 - Walkthrough
 - Inspection
 - Peer Review

2. Static Analysis

- Analyzing code without execution
- Tools detect:
 - Syntax errors
 - Dead code
 - Security issues

3. Formal Verification

- Uses mathematical techniques
- Proves correctness using logic

4. Prototyping

- Early model of system to validate requirements

◇ 3. SRS Verification (Software Requirement Specification)

Definition:

Process of checking whether the **SRS document is correct, complete, and consistent**

Characteristics of Good SRS:

- Correct
- Complete
- Consistent
- Unambiguous
- Verifiable
- Traceable

Verification Activities:

- Requirement reviews
- Stakeholder validation
- Checking ambiguities

Example:

Wrong: “System should be fast”

Correct: “System should respond within 2 seconds”

◇ 4. Source Code Reviews

Definition:

Systematic examination of source code to find errors before execution

Objectives:

- Detect bugs early
- Improve code quality
- Maintain coding standards

Types:

1. Peer Review
2. Walkthrough
3. Inspection

What is Checked:

- Logic errors
- Coding standards
- Security issues
- Performance issues

Example:

Checking loop conditions, variable usage, null pointer issues

◇ **5. User Documentation Verification**

Definition:

Ensuring that user manuals and guides are **accurate and understandable**

What is Verified:

- Correct instructions
- Clarity of language
- Consistency with software
- Completeness

Methods:

- Review by technical experts
- Testing documentation with users

Example:

If software has a “Login” button → documentation must clearly explain login steps

◇ 6. Software Project Audit

Independent evaluation of software processes, activities, and deliverables

Objectives:

- Ensure compliance with standards
- Improve process quality
- Identify risks

Types:

1. Internal Audit
2. External Audit

Areas Covered:

- Documentation
- Coding standards
- Testing processes
- Project management

◇ 7. Tailoring Software Quality Assurance (SQA)**Definition:**

Customizing SQA activities based on project needs

Why Tailoring is Needed:

- Different projects have different requirements
- Avoid unnecessary overhead
- Optimize cost and time

Factors:

- Project size
- Complexity
- Risk level
- Team experience

Example:

Small project → fewer reviews

Large project → strict inspections and audits

◇ 8. Reviews

Process of evaluating software artifacts by a group

Types:

- Informal Review
- Formal Review

Benefits:

- Early error detection
- Knowledge sharing
- Improved quality

◇ 9. Walkthrough**Definition:**

Author explains the document/code step-by-step to reviewers

Features:

- Informal
- No strict rules
- Focus on understanding

Process:

1. Author presents
2. Team asks questions
3. Feedback is given

Advantage:

- Easy and quick

◇ 10. Inspection

Formal and structured review process

Features:

- Highly systematic
- Roles assigned:
 - Moderator
 - Reader
 - Recorder

Steps:

1. Planning
2. Overview
3. Preparation
4. Inspection meeting
5. Rework
6. Follow-up

Advantage:

- Very effective in finding defects

11. Configuration Audit

Definition:

Verification that all software components are correct and properly maintained

Types:

1. Functional Configuration Audit (FCA)

- Checks if system meets requirements

2. Physical Configuration Audit (PCA)

- Checks actual product matches design

Objectives:

- Ensure consistency
- Verify version control
- Validate system completeness